WEEK 1 · LESSON 4 OF 24 · HANDOUT

Logistic Regression: Predicting Yes or No

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=yIYKR4sgzI8 — open the lesson online to watch inside the course.

🎓 **Go deeper:** Harvard CS50 AI, Lecture 4: Learning (supervised, reinforcement, unsupervised) — https://www.youtube.com/watch?v=E4M_IQG0d9g

🎓 **Go deeper:** freeCodeCamp: Machine Learning with Python and scikit-learn — full course — https://www.youtube.com/watch?v=hDKCxebp88A

Your goal in this lesson: Train a logistic regression classifier, turn its output into probabilities and decisions, and explain why accuracy alone can mislead.

The problem: will the loan be repaid?

Mobile-money lenders in Rwanda and across Africa give small loans in seconds, using data instead of paperwork. The model must answer a **yes/no** question: will this borrower **default** (fail to repay)? That is **classification**.

Run the loans cell. It creates 2,000 past borrowers. `defaulted = 1` means the loan was not repaid.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

About **21%** of borrowers defaulted.

From a line to a probability

Linear regression can give any number, like −3 or 250. For yes/no we want a **probability** between 0 and 1. Logistic regression computes a straight-line score, then squeezes it through the **sigmoid** (S-shaped) curve:

> *probability = 1 ÷ (1 + e^(−score))*

A very negative score gives a probability near 0, a score of 0 gives 0.5, a large score gives nearly 1. Despite its name, logistic regression is a **classification** model.

Train it

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

clf = LogisticRegression().fit(X_train_s, y_train)
print("Accuracy:", accuracy_score(y_test, clf.predict(X_test_s)))
```

`stratify=y` keeps the same share of defaulters (21%) in both the training and test sets — important whenever one class is smaller.

You should get about **0.84** accuracy.

Is 84% good? Check the baseline

```
from sklearn.dummy import DummyClassifier

lazy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print("Baseline accuracy:", accuracy_score(y_test, lazy.predict(X_test)))
```

A "model" that says **nobody ever defaults** scores about **0.79** — and is useless, because it never catches a single bad loan. Our model is better, but only by 5 points. In Week 3 you will learn measures (precision, recall) that show what really matters.

Probabilities, not just labels

```
new = pd.DataFrame({
    "income_rwf":      [150_000, 400_000],
    "loan_rwf":        [250_000,  50_000],
    "months_customer": [3, 40],
    "late_payments":   [2, 0],
    "salaried":        [0, 1],
})
print(clf.predict_proba(scaler.transform(new))[:, 1].round(3))
```

The first applicant (low income, big loan, new customer, two late payments) gets a default probability of about **0.92**; the second about **0.001**. `predict()` simply says "1" when the probability is above **0.5** — but a lender can choose another **threshold**, for example refuse above 0.3 (Lesson 15).

Reading the coefficients

```
print(pd.Series(clf.coef_[0], index=X.columns).round(2))
```

Because the features are standardised, the sizes can be compared:

- **income_rwf** is negative: higher income → lower risk.
- **loan_rwf** and **late_payments** are positive: bigger loans and past lateness → higher risk.
- **months_customer** and **salaried** are negative: loyal and salaried customers are safer.

These directions make business sense — a good check that the model learned something real.

More than two classes

Logistic regression also handles several classes (for example: low, medium, high risk). scikit-learn does this automatically when y has more than two values.

Key terms

- **Classification** — predicting a category.
- **Logistic regression** — a linear score passed through the sigmoid to give a probability.
- **Sigmoid** — S-shaped curve that turns any number into 0–1.
- **predict_proba** — the predicted probability of each class.
- **Threshold** — the probability above which we say "yes" (0.5 by default).
- **Stratify** — keep class shares the same in train and test.

Remember this: Logistic regression gives a probability for each case; accuracy must always be compared with the "most frequent class" baseline.

Name: Date:

Activity

Task 1

Train without scaling (use `LogisticRegression(max_iter=1000)`). Is the accuracy very different? Are the coefficients easier or harder to compare?

Task 2

Find the five test borrowers with the highest predicted default probability. Look at their features — do they make sense?

Task 3

Count how many test borrowers have a probability above 0.3, and how many above 0.5.

Task 4

Explain to a loan officer, in three sentences, why 79% accuracy can be useless.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does logistic regression predict?
   - A. A straight line only
   - B. Clusters
   - C. Any number such as a rent
   - D. The probability of a class, used to classify
2. What does the sigmoid function do?
   - A. Draws a straight line
   - B. Squeezes any score into a probability between 0 and 1
   - C. Removes outliers
   - D. Sorts data
3. A model that predicts 'no default' for everyone gets 79% accuracy. Why is that useless?
   - A. It is too slow
   - B. It uses too many features
   - C. 79% is too low
   - D. It never catches a single defaulter
4. What does stratify=y do in train_test_split?
   - A. Removes the target
   - B. Scales the data
   - C. Sorts the rows
   - D. Keeps the same share of each class in train and test
5. By default, when does predict() return 1?
   - A. When the probability is above 0.9
   - B. When the probability is above 0.5
   - C. When income is high
   - D. Always
6. The coefficient for late_payments is positive. What does it mean?
   - A. The feature is useless
   - B. The data is wrong
   - C. More late payments → higher predicted default risk
   - D. Late payments lower risk

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.